

# AAA

## Ejercicio 1

**Enunciado:** Crea una función que reciba como argumento una variable tipo `date` y devuelva un número: 1 si la fecha recibida es igual o posterior a la fecha actual, 0 en cualquier otro caso.

**Solución:**

SQL

```
DELIMITER //
```

```
CREATE FUNCTION verificar_fecha(p_fecha DATE)
RETURNS INT
DETERMINISTIC
BEGIN
    IF p_fecha >= CURDATE() THEN
        RETURN 1;
    ELSE
        RETURN 0;
    END IF;
END //
```

```
DELIMITER ;
```

## Ejercicio 2

**Enunciado:** Se necesita asegurar que para un mismo código de vuelo y fecha no exista el mismo pasajero con una reserva de 2 o más asientos. Implementa un trigger que: 2.1. Evite hacer un `insert` para que no se produzca esa situación. No es necesario hacer el trigger para el `update`. 2.2. Genere un error con el mensaje "ya existe pasajero con asiento en ese vuelo".

**Solución:**

SQL

```
DELIMITER //
```

```
CREATE TRIGGER trg_limite_asientos_pasajero
BEFORE INSERT ON reserva_asientos
FOR EACH ROW
BEGIN
    DECLARE v_cantidad_reservas INT;

    SELECT COUNT(*) INTO v_cantidad_reservas
    FROM reserva_asientos
    WHERE cod_vuelo = NEW.cod_vuelo
        AND fecha = NEW.fecha
        AND id_pasajero = NEW.id_pasajero;

    IF v_cantidad_reservas >= 1 THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'ya existe pasajero con asiento en ese vuelo';
    END IF;
END //
```

```
DELIMITER ;
```

## Ejercicio 3

**Enunciado:** Realizar un procedimiento almacenado para reservar asientos, debe recibir los

siguientes argumentos: v\_id\_pasajero int (entrada), v\_cod\_vuelo char(7) (entrada), v\_cod\_asiento char(3) (entrada), v\_fecha date (entrada), v\_resultadoOK numeric(1) (salida), v\_mensaje\_error varchar(100) (salida).

El procedimiento debe realizar lo siguiente: 3.1. Validar v\_fecha con la función desarrollada en el ejercicio 1, si no supera la validación pondrá v\_resultadoOK=0 y

v\_mensaje\_error='Fecha incorrecta' y no continuará el procedimiento. 3.2.

Comenzar una transacción. 3.3. Bloquear todas las filas de la tabla reserva\_asientos que

tenga el vuelo v\_cod\_vuelo y fecha v\_fecha. 3.4. Borrar cualquier reserva que pudiera tener el pasajero v\_id\_pasajero en el vuelo v\_cod\_vuelo y en la fecha v\_fecha. 3.5. Realizar

una consulta para determinar si existe algún pasajero con reserva de asiento con el vuelo

v\_cod\_vuelo, fecha v\_fecha y asiento v\_cod\_asiento. 3.5.1. Si existe pasajero y es

distinto a v\_id\_pasajero pondrá v\_resultadoOK=0 y v\_mensaje\_error="El asiento está reservado por otro pasajero" y no continuará el procedimiento (rollback). 3.6. Insertar en la tabla reserva\_asientos los datos de la reserva de asientos

recibida. 3.7. Confirmar la transacción si no ha habido errores, rechazar los cambios de la

transacción si ha habido errores de validación.

### Solución:

#### SQL

```
DELIMITER //
```

```
CREATE PROCEDURE reservar_asientos (  
    IN v_id_pasajero INT,  
    IN v_cod_vuelo CHAR(7),  
    IN v_cod_asiento CHAR(3),  
    IN v_fecha DATE,  
    OUT v_resultadoOK NUMERIC(1),  
    OUT v_mensaje_error VARCHAR(100)  
)  
BEGIN  
    DECLARE v_pasajero_existente INT DEFAULT NULL;  
  
    -- 3.1. Validar fecha  
    IF verificar_fecha(v_fecha) = 0 THEN  
        SET v_resultadoOK = 0;  
        SET v_mensaje_error = 'Fecha incorrecta';  
    ELSE  
        -- 3.2. Comenzar una transacción  
        START TRANSACTION;  
  
        -- 3.3. Bloquear todas las filas del vuelo y fecha  
        SELECT * FROM reserva_asientos  
        WHERE cod_vuelo = v_cod_vuelo AND fecha = v_fecha  
        FOR UPDATE;  
  
        -- 3.4. Borrar cualquier reserva previa del pasajero en ese vuelo y  
fecha  
        DELETE FROM reserva_asientos  
        WHERE cod_vuelo = v_cod_vuelo  
            AND fecha = v_fecha  
            AND id_pasajero = v_id_pasajero;  
  
        -- 3.5. Determinar si existe algún pasajero en ese asiento específico  
        -- Se utiliza la asignación directa a la variable para evitar errores  
"No data" si está vacío  
        SET v_pasajero_existente = (  
            SELECT id_pasajero  
            FROM reserva_asientos
```

```

        WHERE cod_vuelo = v_cod_vuelo
          AND fecha = v_fecha
          AND cod_asiento = v_cod_asiento
        LIMIT 1
    );

    -- 3.5.1. Validar si el asiento está ocupado por otro pasajero
    IF v_pasajero_existente IS NOT NULL AND v_pasajero_existente !=
v_id_pasajero THEN
        SET v_resultadoOK = 0;
        SET v_mensaje_error = 'El asiento está reservado por otro pasajero';

        -- 3.7. Rechazar cambios (rollback)
        ROLLBACK;
    ELSE
        -- 3.6. Insertar los datos de la nueva reserva
        INSERT INTO reserva_asientos (cod_vuelo, fecha, cod_asiento,
id_pasajero)
        VALUES (v_cod_vuelo, v_fecha, v_cod_asiento, v_id_pasajero);

        -- 3.7. Confirmar la transacción
        SET v_resultadoOK = 1;
        SET v_mensaje_error = 'Reserva confirmada';
        COMMIT;
    END IF;

END IF;
END //

DELIMITER ;

```

## Ejercicio 1

**Enunciado:** Crea una función que reciba como argumento una variable tipo date y devuelva un número: 1 si la fecha recibida es igual o posterior a la fecha actual, 0 en cualquier otro caso.

**Solución:**

SQL

```
DELIMITER //
```

```
CREATE FUNCTION verificar_fecha(p_fecha DATE)
RETURNS INT
DETERMINISTIC
BEGIN
    IF p_fecha >= CURDATE() THEN
        RETURN 1;
    ELSE
        RETURN 0;
    END IF;
END //
```

```
DELIMITER ;
```

## Ejercicio 2

**Enunciado:** Se necesita asegurar que para una misma sala y fecha no exista el mismo cliente con una reserva de 2 o más butacas. Implementa un trigger que: 2.1. Evite hacer un insert para que no se produzca esa situación. No es necesario hacer el trigger para el update. 2.2. Genere un error con el mensaje "ya existe cliente con butaca reservada en esa sala".

**Solución:**

SQL

```
DELIMITER //
```

```
CREATE TRIGGER trg_limite_butacas_cliente
BEFORE INSERT ON reserva_butacas
FOR EACH ROW
BEGIN
    DECLARE v_cantidad_reservas INT;

    SELECT COUNT(*) INTO v_cantidad_reservas
    FROM reserva_butacas
    WHERE cod_sala = NEW.cod_sala
        AND fecha = NEW.fecha
        AND id_cliente = NEW.id_cliente;

    IF v_cantidad_reservas >= 1 THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'ya existe cliente con butaca reservada en esa sala';
    END IF;
END //
```

```
DELIMITER ;
```

## Ejercicio 3

**Enunciado:** Realizar un procedimiento almacenado para reservar butacas, debe recibir los siguientes argumentos: v\_id\_cliente int (entrada), v\_cod\_sala numeric(2,0) (entrada), v\_cod\_butaca char(3) (entrada), v\_fecha date (entrada), v\_resultadoOK

numeric(1) (salida), v\_mensaje\_error varchar(100) (salida).

El procedimiento debe realizar lo siguiente: 3.1. Validar v\_fecha con la función desarrollada en el ejercicio 1, si no supera la validación pondrá v\_resultadoOK=0 y

v\_mensaje\_error='Fecha incorrecta' y no continuará el procedimiento. 3.2.

Comenzar una transacción. 3.3. Bloquear todas las filas de la tabla reserva\_butacas que tenga

la sala v\_cod\_sala y fecha v\_fecha. 3.4. Borrar cualquier reserva que pudiera tener el cliente

v\_id\_cliente en la sala v\_cod\_sala y en la fecha v\_fecha. 3.5. Realizar una consulta para determinar si existe algún cliente con reserva de butaca en la sala v\_cod\_sala, fecha v\_fecha

y butaca v\_cod\_butaca. 3.5.1. Si existe cliente y es distinto a v\_id\_cliente pondrá

v\_resultadoOK=0 y v\_mensaje\_error="La butaca está reservado por otro cliente" y no continuará el procedimiento. 3.6. Insertar en la tabla reserva\_butacas los

datos de la reserva recibida (v\_cod\_sala, v\_fecha, v\_cod\_butaca y v\_id\_cliente).

3.7. Confirmar la transacción si no ha habido errores, rechazar los cambios de la transacción si ha habido errores de validación.

### Solución:

#### SQL

```
DELIMITER //
```

```
CREATE PROCEDURE reservar_butacas (
    IN v_id_cliente INT,
    IN v_cod_sala NUMERIC(2,0),
    IN v_cod_butaca CHAR(3),
    IN v_fecha DATE,
    OUT v_resultadoOK NUMERIC(1),
    OUT v_mensaje_error VARCHAR(100)
)
BEGIN
    DECLARE v_cliente_existente INT DEFAULT NULL;

    -- 3.1. Validar fecha
    IF verificar_fecha(v_fecha) = 0 THEN
        SET v_resultadoOK = 0;
        SET v_mensaje_error = 'Fecha incorrecta';
    ELSE
        -- 3.2. Comenzar una transacción
        START TRANSACTION;

        -- 3.3. Bloquear todas las filas de la sala y fecha
        SELECT * FROM reserva_butacas
        WHERE cod_sala = v_cod_sala AND fecha = v_fecha
        FOR UPDATE;

        -- 3.4. Borrar cualquier reserva previa del cliente en esa sala y fecha
        DELETE FROM reserva_butacas
        WHERE cod_sala = v_cod_sala
            AND fecha = v_fecha
            AND id_cliente = v_id_cliente;

        -- 3.5. Determinar si existe algún cliente en esa butaca específica
        SET v_cliente_existente = (
            SELECT id_cliente
            FROM reserva_butacas
            WHERE cod_sala = v_cod_sala
                AND fecha = v_fecha
                AND cod_butaca = v_cod_butaca
            LIMIT 1
        );
    END IF;
END;
```

```

        -- 3.5.1. Validar si la butaca está ocupada por otro cliente
        IF v_cliente_existente IS NOT NULL AND v_cliente_existente !=
v_id_cliente THEN
            SET v_resultadoOK = 0;
            SET v_mensaje_error = 'La butaca está reservado por otro cliente';

            -- 3.7. Rechazar cambios (rollback)
            ROLLBACK;
        ELSE
            -- 3.6. Insertar los datos de la nueva reserva
            INSERT INTO reserva_butacas (cod_sala, fecha, cod_butaca,
id_cliente)
            VALUES (v_cod_sala, v_fecha, v_cod_butaca, v_id_cliente);

            -- 3.7. Confirmar la transacción
            SET v_resultadoOK = 1;
            SET v_mensaje_error = 'Reserva confirmada';
            COMMIT;
        END IF;

    END IF;
END //

DELIMITER ;

```